

Introdução a Redes Neurais e Redes Convolucionais

Trabalho Individual — Tarefas 1, 2 e 3

Aluno: Lucas Medeiros

Disciplina: Aprendizado Profundo — PPgTI/IMD/UFRN

Docente: Prof. Josenalde Oliveira

Setembro de 2026

Sumário

Tarefa 1 — Estudo Dirigido: TensorFlow Playground	3
Tarefa 2 — Otimização de Hiperparâmetros do MNIST (Optuna)	7
Tarefa 3 — CNNs (PyTorch) para o Multiprova Corretor (EMNIST)	12
Links dos Notebooks e Repositório	15

Tarefa 1 — Estudo Dirigido: TensorFlow Playground

Todos os experimentos abaixo foram executados de fato no [TensorFlow Playground](#), interagindo com a interface (troca de hiperparâmetros, treino, reset) e registrando os valores reais de época, perda de treino/teste e o formato da fronteira de decisão observada.

a) Padrões aprendidos por uma NN (configuração padrão)

Configuração: dataset *Circle*, 2 features (X1, X2), 2 camadas ocultas (4 e 2 neurônios), ativação **tanh**, taxa de aprendizagem 0.03, sem regularização, ruído 0, batch size 10.

Ao treinar, a rede converge rapidamente: em **699 épocas** obtive perda de teste = **0.002** e perda de treino = **0.001**, com uma fronteira de decisão circular suave, coerente com a distribuição dos dados. Isso confirma o comportamento descrito: os neurônios da primeira camada oculta aprendem padrões simples (retas/curvas separando parcialmente o espaço) e a segunda camada combina essas saídas para formar a fronteira circular final — quanto mais camadas/neurônios, mais complexos os padrões que a rede pode compor.

b) Função de ativação

Mantendo a arquitetura de (a), troquei apenas a ativação:

Ativação	Épocas	Perda teste	Perda treino	Fronteira de decisão
Tanh (baseline)	699	0.002	0.001	curva suave (circular)
ReLU	621	0.001	0.000	poligonal/angular, com "quinas"
Linear	1918 (sem melhora)	0.514	0.492	nenhuma fronteira útil

Com **ReLU** a rede convergiu um pouco mais rápido que com tanh e chegou a uma perda ligeiramente menor, mas a fronteira deixou de ser suave: como o ReLU é linear por partes, a região de decisão vira um polígono (faces retas), enquanto o tanh gera fronteiras curvas por ser uma função suave/saturante.

Com **Linear**, mesmo depois de quase 2000 épocas o erro não sai de ~0.5 (equivalente a chute aleatório). Isso é esperado: uma composição de camadas totalmente lineares equivale, algebricamente, a uma única camada linear (não importa a profundidade), e um classificador linear não consegue separar um padrão circular, que não é linearmente separável.

c) Taxa de aprendizagem (learning rate)

Mantendo tanh e demais parâmetros fixos:

Learning rate	Épocas	Perda teste	Perda treino	Observação
0.03 (baseline)	699	0.002	0.001	convergência estável
0.00001 (muito baixa)	841	0.512	0.493	quase não aprende
10 (muito alta)	790	0.768	0.816	diverge/instável

Taxa de aprendizado baixa demais faz o treinamento ser extremamente lento (paralisia de aprendizado); alta demais faz o gradiente descendente "pular" o mínimo e divergir, tornando o treino instável — a perda pode até aumentar em vez de diminuir.

d) Risco de mínimos locais (1 camada oculta, 3 neurônios)

Arquitetura reduzida para 1 camada oculta com 3 neurônios (tanh, lr 0.03). Treinei a rede 4 vezes, resetando os pesos entre cada tentativa:

Tentativa	Épocas até parar	Perda teste final	Resultado
1	1019	0.011	convergiu bem
2	1121	0.009	convergiu bem
3	1209	0.009	convergiu bem
4	1152	0.268	preso em mínimo local

Na tentativa 4, a curva de perda estabiliza num platô alto e a rede nunca escapa dele — ilustra o risco de mínimos locais: com poucos neurônios/pesos, a superfície de erro tem menos "rotas" de escape.

e) Poucos neurônios (2 neurônios, 1 camada)

Reduzindo ainda mais, para 2 neurônios em 1 camada oculta: em duas tentativas independentes, a rede ficou presa exatamente na mesma perda de teste = **0.231**, com a mesma fronteira (faixa diagonal), mesmo após mais de 1200 épocas em ambas. Aqui não é "azar" de inicialização (como em d) — a arquitetura simplesmente não tem capacidade suficiente para representar a fronteira circular.

f) Rede maior (8 neurônios, 1 camada)

Com 8 neurônios numa única camada oculta, repeti o treino duas vezes: convergência em **651 e 659 épocas**, ambas com perda de teste $\approx$ **0.008–0.009**, sem sinal de estagnação. Confirma que redes com maior capacidade tendem a não ficar presas em mínimos locais ruins.

g) Dataset espiral, 4 camadas ocultas $\times$ 8 neurônios

- **Baseline** (lr = 0.03, sem regularização): perda de teste = 0.003 aos 812 épocas, com um trecho de platô antes de despencar.
- **Learning rate alta (0.1) + regularização L2 (0.01)**: perda travada em 0.480 entre as épocas 319 e 858, com boa parte dos pesos empurrados perto de zero.
- **Learning rate alta (0.1) + regularização L1 (0.01)**: em 371 épocas a perda estagnou em 0.501 (chute aleatório) e praticamente todos os pesos foram zerados (esparsificação da L1).

No dataset espiral, learning rate alta **junto com** regularização forte é contraproducente — a regularização penaliza os pesos exatamente quando a rede mais precisa deles para modelar a espiral.

Exercícios de fixação

1) Três funções de ativação populares:

- **Sigmoide:** $\sigma(z) = 1/(1+e^{-z})$, saída entre 0 e 1.
- **Tanh:** $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, saída entre -1 e 1, centrada em zero.
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$.

2) MLP com 10 neurônios de entrada, 1 camada oculta com 50 neurônios, saída com 3 neurônios, tudo ReLU (notação de Géron, *Mãos à Obra*, 2ª ed., p. 219–220; m = nº de instâncias no lote):

- a) X: formato (m, 10).
- b) Camada oculta: **Wh** formato (10, 50), **bh** formato (50,).
- c) Camada de saída: **Wo** formato (50, 3), **bo** formato (3,).
- d) Y: formato (m, 3).
- e) $Y = \text{ReLU}(\text{ReLU}(X \cdot W_h + b_h) \cdot W_o + b_o)$

3) Neurônios/ativação na camada de saída:

- **Spam ou não spam:** 1 neurônio, ativação sigmoide.
- **MNIST:** 10 neurônios, ativação softmax.
- **Cotação do dólar amanhã (regressão):** 1 neurônio, sem ativação (linear).

4) Hiperparâmetros e overfitting:

Hiperparâmetros: nº de camadas, neurônios por camada, ativação, learning rate, otimizador, épocas, batch size, regularização (L1/L2, dropout), inicialização dos pesos.

Contra overfitting: reduzir complexidade do modelo; L1/L2; dropout; early stopping; mais dados/data augmentation; menos épocas; normalizar melhor os dados; validação cruzada.

Tarefa 2 — Otimização de Hiperparâmetros do MNIST (Optuna)

Notebook: tarefa2_mnist_optuna.ipynb (executado localmente, Python 3.13 / Windows, TensorFlow 2.21.0, CPU). Base: notebook do professor mnist_keras.ipynb.

1. Metodologia

Utilizou-se a biblioteca **Optuna** (algoritmo TPE, direction="maximize") para buscar a melhor combinação de hiperparâmetros de uma MLP (rede densa) para classificação do MNIST, otimizando a acurácia de validação (10% do treino). Espaço de busca:

- n_layers: número de camadas densas ocultas (1 a 3)
- units: número de neurônios por camada oculta (32, 64 ou 128)
- activation: função de ativação (relu ou tanh)
- drop_rate: taxa de dropout (0.0 a 0.5)
- learning_rate: taxa de aprendizado do Adam (1e-4 a 1e-2, escala log)
- batch_size: tamanho do lote (32, 64 ou 128)

Cada trial treinou o modelo por 8 épocas, com 20 trials no total.

2. Resultados da busca (20 trials)

Trial	Acc. val.	batch	n_layers	units	ativ.	dropout	lr
0	0.9692	128	1	64	relu	0.499	0.00206
1	0.9650	64	1	32	tanh	0.029	0.00478
2	0.9715	128	3	32	relu	0.017	0.00565
3	0.9645	32	1	64	tanh	0.389	0.00394
4	0.9648	64	2	64	relu	0.458	0.00046
5	0.9228	64	3	32	relu	0.492	0.00062
6	0.9798	32	1	128	relu	0.254	0.00052
7	0.9808	128	2	128	relu	0.1125	0.00243
8	0.9775	32	1	128	tanh	0.230	0.00318
9	0.9753	128	1	64	relu	0.295	0.00313
10	0.9615	128	2	128	tanh	0.132	0.00018
11	0.9805	32	2	128	relu	0.188	0.00096
12	0.9802	32	2	128	relu	0.125	0.00135
13	0.9630	32	2	128	relu	0.143	0.00972
14	0.9782	128	2	128	relu	0.187	0.00117
15	0.9782	32	3	128	relu	0.093	0.00019
16	0.9803	128	2	128	relu	0.333	0.00107
17	0.9793	128	2	128	relu	0.066	0.00191
18	0.9735	32	3	128	tanh	0.209	0.00034

Trial	Acc. val.	batch	n_layers	units	ativ.	dropout	lr
19	0.9420	64	2	32	relu	0.171	0.00011

Melhor trial: nº 7, acc. validação **0.9808**: batch_size=128, n_layers=2, units=128, activation=relu, drop_rate=0.1125, learning_rate=0.002429.

3. Modelo final e avaliação no conjunto de teste

O modelo final foi reconstruído com os melhores hiperparâmetros do trial 7 e treinado por 25 épocas. Arquitetura: Flatten(784) → Dense(128, relu) → Dropout(0.1125) → Dense(128, relu) → Dropout(0.1125) → Dense(10, softmax). **Total de parâmetros treináveis: 118.282** (≈462 KB).

Resultado no conjunto de TESTE (10.000 imagens): acurácia = **0.9788**, perda = **0.1058**.

classification_report (teste):

Classe	Precisão	Recall	F1-score	Suporte
0	0.99	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.98	0.98	1032
3	0.97	0.98	0.98	1010
4	0.99	0.97	0.98	982
5	0.96	0.98	0.97	892
6	0.98	0.98	0.98	958
7	0.97	0.98	0.98	1028
8	0.99	0.96	0.97	974
9	0.96	0.98	0.97	1009
acurácia geral			0.98	10000

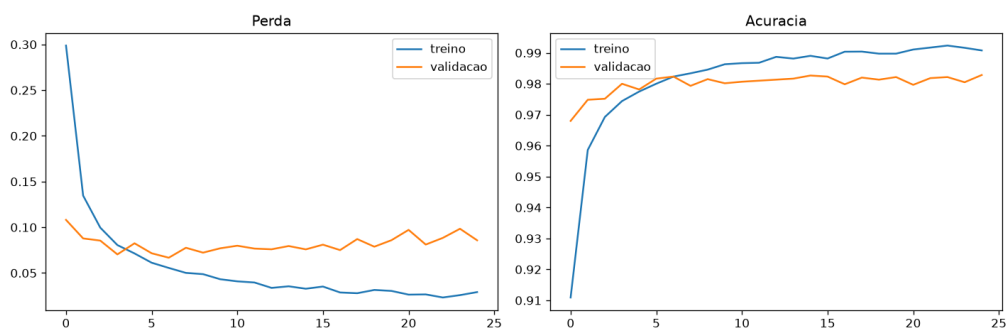


Figura 1 — Curvas de perda e acurácia (treino x validação) ao longo das 25 épocas.

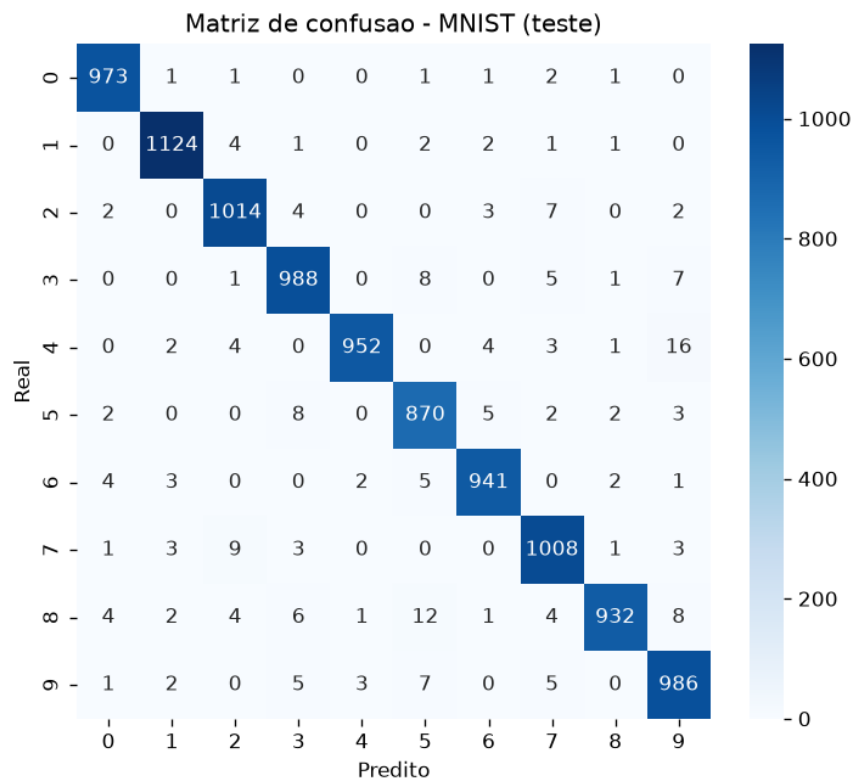


Figura 2 — Matriz de confusão do modelo final no conjunto de teste.

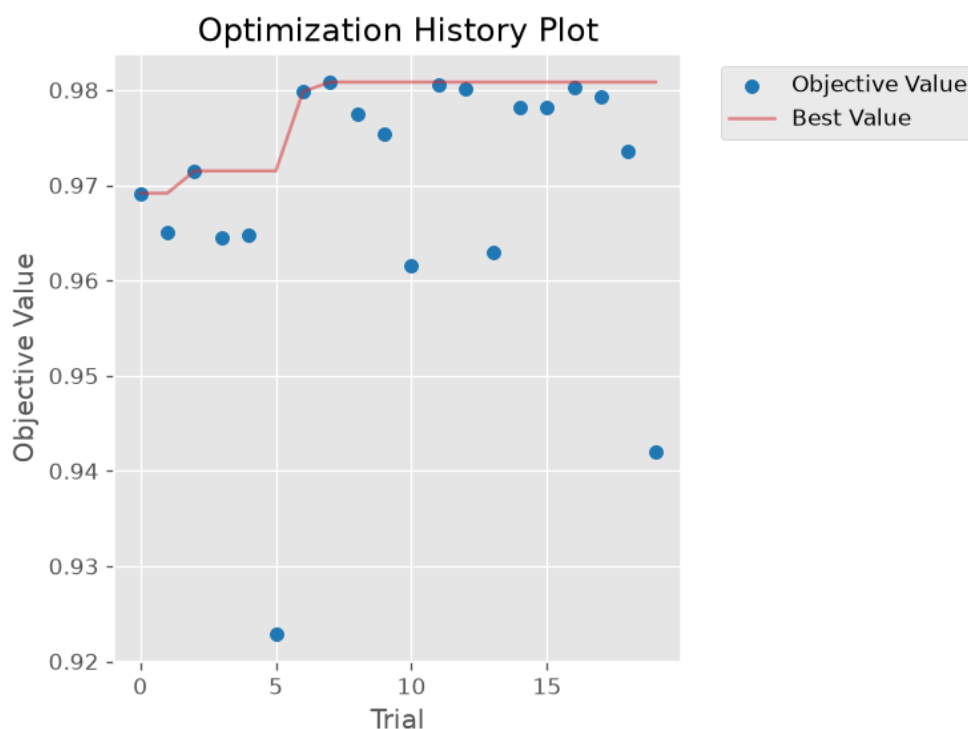


Figura 3 — Histórico de otimização do Optuna (valor objetivo por trial).

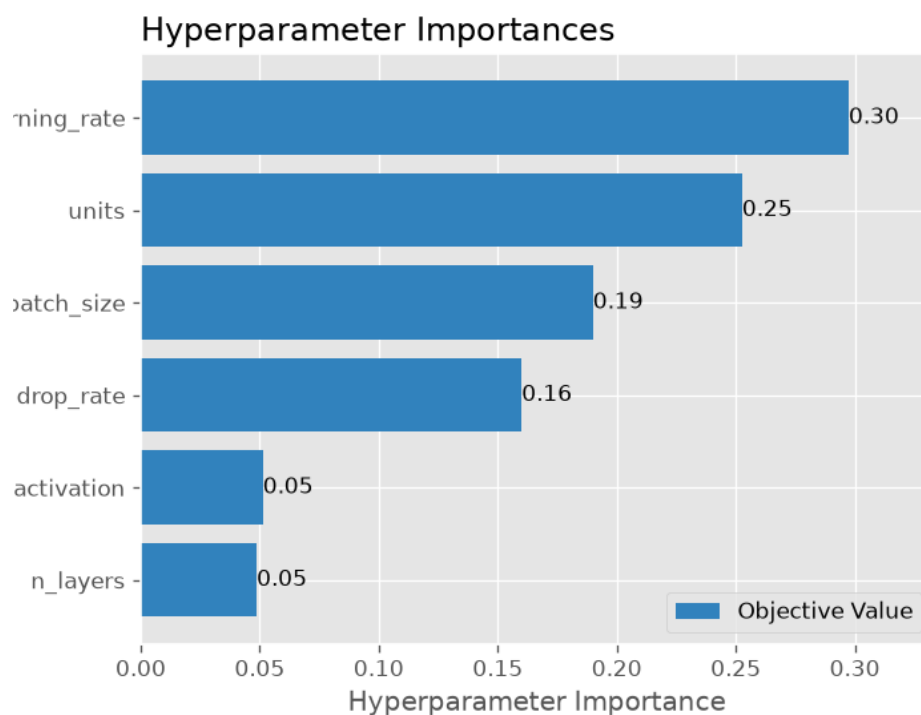


Figura 4 — Importância de cada hiperparâmetro na busca do Optuna.

4. Comentário sobre o ajuste final e sua qualidade

Observando os 20 trials, o padrão é claro: os melhores resultados (acc. validação > 0.978) concentram-se em `n_layers=1` ou `2`, `units=128` e `activation=relu`, com `learning_rate` entre $\sim 5e-4$ e $\sim 2.5e-3$, e `batch_size` de 32 ou 128. Isso é coerente com a teoria:

- **Mais unidades por camada (128) e poucas camadas** já bastam para o MNIST, um problema relativamente simples; `n_layers=3` não trouxe ganho consistente (trials 2, 5, 15, 18 variam bastante), sinal de que a profundidade extra não é bem aproveitada em só 8 épocas de busca.
- **ReLU superou tanh** de forma quase sistemática entre os melhores trials — a `tanh` (trials 1, 3, 8, 10, 18) tende a saturar mais e converge mais devagar nesse regime de poucas épocas.
- **Learning rates muito baixos** ($\sim 1e-4$, trials 10, 15, 19) e **muito altos** ($\sim 1e-2$, trial 13) tiveram desempenho pior — o primeiro caso por não convergir a tempo, o segundo por instabilidade no treino. A faixa "doce" ficou entre $\sim 5e-4$ e $\sim 2.5e-3$.
- O **dropout** ótimo encontrado (0.1125) é moderado-baixo — sugere que, no MNIST, uma regularização mais leve já é suficiente.

A busca convergiu para uma região estável de hiperparâmetros: os trials 7, 11, 12, 16 (todos com `units=128`, `n_layers=2`, `relu`, `learning rate` entre 0.001 e 0.0024) têm desempenho muito próximo (0.980–0.981), o que indica um ótimo estável e não um pico isolado de ruído estatístico. O ganho sobre a configuração mais fraca da busca (trial 5, 0.9228) é de mais de 5 pontos percentuais de acurácia de validação, obtido de forma sistemática via busca automática — e o resultado final no teste (0.9788) confirma que o ajuste generaliza bem, sem overfitting aos dados de validação usados durante a busca.

Tarefa 3 — CNNs (PyTorch) para o Multiprova Corretor (EMNIST)

Notebook: `tarefa3_cnn_emnist_pytorch.ipynb` (executado localmente, Python 3.13 / Windows, PyTorch 2.14.0, CPU). Reprodução, em PyTorch, da melhor arquitetura de CNN de Silva Filho et al. (2022) para o app Multiprova Corretor, aplicada a 3 subproblemas construídos a partir do EMNIST.

1. Arquitetura

CNNMultiprova: 4 blocos Conv2d(3x3) → ReLU → MaxPool2d(2) com 32→64→64→64 filtros, seguidos de Flatten → Linear(256, n_classes). Entrada: imagens EMNIST redimensionadas para 32×32, 1 canal. Treino: Adam (lr=1e-3), CrossEntropyLoss, batch size 128, até 12 épocas com early stopping (paciência 3, monitorando perda de validação).

2. Subproblemas e resultados

- **Dígitos (1-5)** — split "digits" do EMNIST (102.000 treino / 18.000 validação / 20.000 teste). Convergiu em 7 épocas. **Acurácia de teste: 0.9975**. classification_report: precisão/recall/f1 = 1.00 em todas as 5 classes.
- **V ou F** — split "letters", filtrando "v" e "f" (8.160 treino / 1.440 validação / 1.600 teste). Convergiu em 11 épocas. **Acurácia de teste: 1.0000** (100%).
- **Letras A-E** — split "letters", filtrando "a" a "e" (20.400 treino / 3.600 validação / 4.000 teste). Convergiu em 9 épocas. **Acurácia de teste: 0.9782**. A letra A teve o menor recall (0.95), provavelmente por confusão visual com outras letras manuscritas parecidas.

Subproblema	Épocas até parar	Acc. teste
Dígitos (1-5)	7	0.9975
V ou F	11	1.0000
Letras (A-E)	9	0.9782

3.1) Tabela: parâmetros treináveis e tamanho dos modelos exportados (.pt)

Modelo	Parâmetros treináveis	Tamanho (MB)
Dígitos (1-5)	93.957	0.363
V ou F	93.186	0.359
Letras (A-E)	93.957	0.363

Os três modelos têm praticamente o mesmo tamanho (a diferença de ~770 parâmetros entre os modelos de 5 classes e o de 2 classes vem só da última camada densa, Linear(256, n_classes)). Isso confirma que o custo do modelo é dominado pelas camadas convolucionais (compartilhadas na arquitetura), não pela camada de saída — os modelos são bem enxutos (< 0.4 MB cada), compatíveis com uso embarcado em um app mobile como o Multiprova Corretor.

3.2) Como o tamanho do modelo poderia ser reduzido?

- **Reduzir a arquitetura:** menos filtros por camada (ex.: 16/32/32/32 em vez de 32/64/64/64) e/ou menos camadas.

- **Quantização:** converter os pesos de float32 para int8 (ou float16), reduzindo o tamanho em até 4x com perda mínima de acurácia.
- **Poda (pruning):** remover pesos/filtros com magnitude próxima de zero, gerando uma rede esparsa.
- **Convoluções separáveis em profundidade** (depthwise separable, como no MobileNet).
- **Destilação de conhecimento:** treinar uma rede "aluno" menor para imitar as saídas dessas CNNs já treinadas.
- **Global Average Pooling** no lugar do Flatten + Linear final.
- **Fatoração de baixo posto** (low-rank factorization) das matrizes de peso.
- **Formatos de exportação mais compactos:** ONNX ou TensorFlow Lite.

Como os modelos aqui já são pequenos (< 0.4 MB), a estratégia mais custo-benefício para um app mobile como o Multiprova Corretor seria **quantização pós-treinamento para INT8** (redução de ~4x quase de graça, sem retreinar) combinada com **exportação para um formato compacto** (ONNX Runtime Mobile ou TFLite), reservando poda/destilação/arquitetura menor para cenários com restrição de memória ainda mais severa.

Links dos Notebooks e Repositório

Repositório GitHub (código, notebooks executados e relatórios):

<https://github.com/lucassmsantoss/aprendizado-profundo>

Notebook Tarefa 2 (Colab/Kaggle):

[\[inserir link após upload do tarefa2_mnist_optuna.ipynb\]](#)

Notebook Tarefa 3 (Colab/Kaggle):

[\[inserir link após upload do tarefa3_cnn_emnist_pytorch.ipynb\]](#)